

BASES PARA DESARROLLAR SOFTWARE REACTIVO

R. A. Ramírez Gómez

7690-22-12700 Universidad Mariano Gálvez

Seminario de Tecnologías de Información

rramirezg18@miumg.edu.gt

Resumen

El presente artículo trata sobre las bases que se deben tener en cuenta para construir software reactivo, lo cual es importante debido a que las aplicaciones actuales deben soportar grandes volúmenes de datos y responder de forma inmediata a los usuarios. Se investigó sobre los cuatro principios establecidos en el Manifiesto Reactivo que son ser responsivo, resiliente, elástico y estar dirigido por mensajes, además de que se abordó la programación reactiva como paradigma que trata los datos como flujos que cambian con el tiempo. Se describieron algunas buenas prácticas específicas como el uso correcto de los operadores de transformación sobre los flujos de datos, el manejo adecuado de errores y la aplicación de pruebas unitarias sobre los componentes reactivos. También se abordó el criterio de cuándo es conveniente utilizar este paradigma y cuándo no. Se concluye que construir software reactivo no depende únicamente de utilizar una librería reactiva, sino de aplicar correctamente estos principios y prácticas durante todo el desarrollo del sistema.

Palabras clave: programación reactiva, resiliencia, flujos de datos, operadores, sistemas reactivos

Desarrollo del tema

Un sistema reactivo es aquel que cumple con cuatro propiedades fundamentales que son responsivo, resiliente, elástico y dirigido por mensajes, tal como lo establece el Manifiesto Reactivo (Bonér et al., 2014). Bonér et al. (2014) explican que ser responsivo significa que el sistema responde en un tiempo razonable siempre que sea posible, además de que esto permite detectar problemas rápidamente y tratarlos de forma efectiva, lo cual genera confianza en el usuario final.

La resiliencia se logra mediante la replicación, el aislamiento y la delegación de fallos entre los distintos componentes del sistema, de manera que un componente pueda fallar y recuperarse sin comprometer al sistema completo (Bonér et al., 2014). Por otro lado, la elasticidad permite que el sistema se mantenga responsivo aunque la carga de trabajo varíe, ya que los recursos se pueden aumentar o disminuir según la demanda sin que existan puntos de contención centralizados. Bonér et al. (2014) también mencionan que la comunicación dirigida por mensajes es la base que permite que los demás principios funcionen en conjunto, ya que el paso de mensajes asíncronos entre componentes asegura un bajo acoplamiento, aislamiento y transparencia de ubicación.

Desde el punto de vista de la programación, Bainomugisha et al. (2013) explican que la programación reactiva es un paradigma que trata los valores como flujos que cambian con el tiempo en lugar de tratarlos como valores estáticos, lo cual facilita el desarrollo de aplicaciones dirigidas por eventos e interactivas. Bainomugisha et al. (2013) agregan que uno de los principales beneficios de este paradigma es que el lenguaje o framework se encarga de gestionar automáticamente las dependencias entre estos valores cambiantes, evitando que el programador tenga que actualizar manualmente cada parte del sistema que se vea afectada por un cambio.

Para comprender cómo se construye software reactivo es necesario conocer sus elementos base. Pragma (2024) señala que la programación reactiva se articula mediante tres elementos fundamentales que son los observables, que funcionan como un canal por donde fluyen los datos y emiten notificaciones cuando ocurren cambios; los observadores, que se interesan por ese flujo y ejecutan

acciones en consecuencia; y las suscripciones, que son el vínculo que une a ambos y maneja la asignación y liberación de recursos. Pragma (2024) agrega que dentro del ecosistema de Reactor existen dos tipos de datos que representan estos flujos, que son Mono, el cual emite cero o un solo elemento, y Flux, el cual emite cero o muchos elementos.

Dos características centrales de este paradigma que se deben tener presentes durante el desarrollo son la propagación de cambios y la asincronía. Platzi (2023) explica que cuando una variable de datos cambia, todas las dependencias de esa variable son notificadas y actualizadas automáticamente, lo cual evita que el programador tenga que propagar los cambios de forma manual. Platzi (2023) agrega que la programación reactiva maneja la asincronía de forma natural, ya que los flujos de eventos pueden ocurrir en cualquier momento y los observables pueden emitir eventos en un hilo y ser observados en otro, lo cual es clave para no bloquear la ejecución del sistema.

Una de las buenas prácticas más importantes al construir software reactivo es el uso adecuado de los métodos de transformación, también conocidos como operadores reactivos. Pragma (2024) explica que estos métodos toman un flujo como entrada y producen otro flujo como salida, permitiendo modificar, filtrar o combinar los datos, y menciona operadores como map para transformar cada elemento, filter para emitir únicamente los elementos que cumplen una condición, y flatMap para transformar y aplanar flujos anidados en uno solo. El uso correcto de estos operadores permite mantener un código declarativo y ordenado en lugar de manejar la lógica de forma manual.

Otra buena práctica fundamental es la aplicación de pruebas unitarias sobre los componentes reactivos. Pragma (2024) señala que estas pruebas permiten verificar el manejo de eventos, las transformaciones de datos, la gestión del estado, el manejo de errores y el comportamiento asíncrono de cada componente, y recomienda utilizar bibliotecas especializadas como Reactor Test para proyectos basados en Reactor. Realizar estas pruebas ayuda a garantizar que cada parte del sistema se comporte de la forma esperada antes de integrarla al resto de la aplicación.

En cuanto al manejo de la concurrencia, una buena práctica consiste en aprovechar la forma en que el paradigma administra los hilos de ejecución. Platzi (2023) explica que la programación reactiva permite manejar de forma eficiente múltiples peticiones con un conjunto limitado de hilos, maximizando su utilización en lugar de asignar un hilo por cada petición como ocurre en el modelo tradicional. Esta característica es la que permite que una aplicación reactiva escale mejor ante una gran cantidad de solicitudes concurrentes. Del mismo modo, Platzi (2023) menciona que este paradigma facilita la tolerancia a fallos, ya que cuando ocurre un error en un flujo de eventos, este puede ser manejado y aislado sin afectar al resto del sistema, lo cual se relaciona directamente con la resiliencia que exige el Manifiesto Reactivo.

Respecto a la implementación en un caso concreto, Platzi (2023) señala que en el ecosistema de Spring se utilizan principalmente las librerías RxJava y Reactor, siendo Reactor la opción preferida por estar basada en Reactive Streams. Para el desarrollo reactivo, Spring proporciona una biblioteca web llamada WebFlux que reemplaza a la biblioteca web tradicional y utiliza por defecto el servidor Netty en lugar de Tomcat, lo cual permite aprovechar el modelo no bloqueante en las aplicaciones web.

Otra buena práctica relacionada con el control de los flujos de datos es el manejo de la contrapresión, la cual se presenta cuando un productor de datos genera información a mayor velocidad de la que un consumidor puede procesar. Bainomugisha et al. (2013) explican que uno de los desafíos de los sistemas basados en flujos es precisamente coordinar la velocidad a la que se producen y se consumen los datos, ya que un manejo inadecuado puede provocar acumulación de información y agotamiento de los recursos del sistema. Por esta razón, una buena práctica al construir software

reactivo es utilizar operadores y mecanismos que permitan al consumidor regular la cantidad de datos que recibe, de modo que el sistema se mantenga estable incluso cuando la carga de trabajo es alta.

El manejo de errores es otro aspecto que forma parte de las buenas prácticas al construir software reactivo. Pragma (2024) señala que dentro de un flujo de datos los observables pueden emitir notificaciones de error y los observadores pueden definir comportamientos específicos para manejar estas situaciones, lo cual permite que el sistema se recupere de fallos y continúe funcionando de forma confiable. Definir de forma explícita cómo se manejarán los errores dentro de la cadena reactiva, en lugar de dejar que se propaguen sin control, es lo que permite que una aplicación se acerque a la resiliencia que exige el Manifiesto Reactivo.

En cuanto al rendimiento, Platzi (2023) explica que la programación reactiva permite manejar eficientemente múltiples peticiones con un conjunto limitado de hilos, lo cual se traduce en una mayor escalabilidad, un arranque más rápido de las aplicaciones y un menor consumo de memoria y CPU. Sin embargo, Platzi (2023) también advierte que este paradigma puede complicar el código y hacerlo menos legible, además de que requiere más tiempo de aprendizaje y puede ser más complejo de probar y depurar.

Finalmente, una buena práctica que refleja criterio profesional es saber cuándo conviene aplicar este paradigma y cuándo no. Platzi (2023) señala que la programación reactiva es especialmente útil en arquitecturas de microservicios y en aplicaciones que deben manejar muchos eventos en tiempo real, pero advierte que en procesos que involucran un uso intensivo de CPU, como la criptografía, la resolución de problemas matemáticos o los algoritmos de ordenación, este paradigma puede complicar el desarrollo sin mejorar significativamente los resultados. Por esta razón, antes de adoptar la programación reactiva se debe evaluar cuidadosamente el contexto y los requisitos del proyecto.

Observaciones y comentarios

Considero que muchas veces se adopta una librería reactiva sin comprender a fondo los principios que la sustentan, lo cual provoca que el código pierda los beneficios de este paradigma. También pude notar que decisiones como aplicar pruebas unitarias sobre los flujos o evaluar si el proyecto realmente necesita ser reactivo no se toman con la frecuencia que deberían, aunque son determinantes para la calidad del sistema.

Conclusiones

1. Un sistema reactivo debe cumplir con los cuatro principios del Manifiesto Reactivo, además de que estos principios se deben aplicar en conjunto y no de forma aislada para obtener los beneficios esperados.
2. El uso correcto de los operadores de transformación y la aplicación de pruebas unitarias sobre los componentes reactivos son prácticas que influyen directamente en la calidad y el mantenimiento del sistema.
3. Adoptar la programación reactiva no siempre es la mejor opción, ya que se debe evaluar el contexto del proyecto para determinar si realmente aporta beneficios frente a un enfoque tradicional.

Referencias

Bainomugisha, E., Lombide Carreton, A., Van Cutsem, T., Mostinckx, S., & De Meuter, W. (2013). A survey on reactive programming. *ACM Computing Surveys*, 45(4), Article 52. <https://doi.org/10.1145/2501654.2501666>

Bonér, J., Farley, D., Kuhn, R., & Thompson, M. (2014). *The reactive manifesto* (v2.0). <https://www.reactivemanifesto.org/>

Platzi. (2023, 26 de junio). *Programación reactiva: guía para desarrolladores*. <https://platzi.com/blog/programacion-reactiva/>

Pragma. (2024, 19 de junio). *Guía de programación reactiva en Java*. <https://www.pragma.co/es/blog/guia-de-programacion-reactiva-en-java>

Código fuente: <https://github.com/rramirezg18/foros/tree/main/foro2>